Honeypot Simulation

Personal Project 01

Tan Jun Xiang

17 April 2026

Table of Contents

1. Overview 3

1.1 Problem 3

1.2 Solution 3

1.3 Goals 3

1.4 Components 3

1.5 Features 4

2. Scope 4

2.1 In Scope 4

2.2 Out of Scope 4

3. Setup & Design 4

3.1 Environment 4

3.2 Operating System and Software 5

3.3 Network Design 5

3.4 Cowrie Design 5

4. Methodology 5

4.1 Virtual Machine Setup 5

4.2 Installation of Cowrie Honeypot 5

4.3 Environment and Cowrie Setup 5

4.4 Cowrie Configuration 6

4.5 Cowrie Execution and Monitoring 6

4.6 Connectivity Verification 6

5. Limitations & Risks 7

6. Testing & Findings 7

6.1 Automated Password Guessing Using Hydra 7

6.2 Manual Common-Credential Testing 9

6.3 Successful Login and Command Execution 12

6.4 Test Summary 13

7. Challenges 14

8. Improvements 14

1. Overview

This project simulates a honeypot environment to observe and analyse unauthorised SSH access attempts. The setup allows safe study of brute-force behaviour, command patterns, and attacker interaction in a controlled environment.

1.1 Problem

The project addresses the need for a safe and controlled environment to observe and analyse unauthorised SSH access attempts. This allows brute-force behaviour, credential attempts, command patterns, and attacker interaction to be studied without performing testing in a real-world environment.

1.2 Solution

Using the Cowrie honeypot on Ubuntu and an attacking Kali machine, the setup provides a controlled environment for capturing and analysing unauthorised SSH access attempts.

1.3 Goals

The goal of this project is to:

- Understand common brute-force and unauthorised access techniques.
- Capture and analyse attacker behaviour and commands.
- Gain hands-on experience with defensive security tools like Cowrie.

1.4 Components

Components	Purpose
Ubuntu 22.04 VM	Hosts Cowrie to capture and log attacker activity.
Kali Linux VM	Acts as an attacker performing SSH login attempts against the honeypot.
Cowrie	Provides a controlled, emulated SSH/Telnet environment and detailed logging of login attempts, commands, and interactions.
Hydra	Used to simulate SSH brute-force attacks.
Oracle VirtualBox	Hosts the two virtual machines used to create the isolated environment.

1.5 Features

- Simulates an SSH/Telnet service using the Cowrie honeypot.
- Captures SSH brute-force attempts (usernames, passwords).
- Logs all commands executed by attackers during sessions.
- Generates structured log files for analysis of attacker behaviour.

2. Scope

2.1 In Scope

- Setting up a honeypot environment using Cowrie on Ubuntu.
- Simulating SSH brute-force attacks from a Kali Linux machine.
- Capturing and logging attacker login attempts and commands.
- Basic analysis of attacker behaviour based on collected logs.

2.2 Out of Scope

- Development of custom scripts or programs for automated attack simulation or log analysis.
- Advanced malware analysis or payload execution.
- Integration with SIEM or large-scale monitoring systems.
- Simulation in a real-world environment.
- Defence beyond observation, such as automated blocking or response.

3. Setup & Design

3.1 Environment

The environment was built using two virtual machines inside Oracle VirtualBox.

- Ubuntu VM (Honeypot): Hosts Cowrie to capture and log attacker activity.
- Kali Linux (Attacker): Acts as an attacker performing SSH login attempts against the honeypot.

3.2 Operating System and Software

- Cowrie requires Python 3.10 or higher, so Ubuntu 22.04 was used as it meets the required version by default.
- Kali Linux was cloned from an existing setup.
- Cowrie was obtained from the official public GitHub repository and installed in Ubuntu VM.
- Required dependencies were installed, including Git, Python, and related libraries.

3.3 Network Design

- Both virtual machines were configured to use a Host-Only Adapter to allow direct communication while maintaining an isolated environment.
- The Ubuntu VM was temporarily configured to use a NAT Network while downloading the required dependencies before being switched back to the Host-Only Adapter.

3.4 Cowrie Design

Cowrie was used for the honeypot as it provides:

- A controlled, emulated SSH/Telnet environment.
- Detailed logging of all login attempts, commands, and interactions.

4. Methodology

4.1 Virtual Machine Setup

1. Ubuntu 22.04 virtual machine was installed and set up for the Cowrie honeypot.
2. An existing Kali Linux setup was cloned to be used as the attacker.
3. Both virtual machines were configured using a Host-Only Adapter to allow direct communication while maintaining an isolated environment.
4. The Ubuntu VM was temporarily set to NAT Network to download dependencies.

4.2 Installation of Cowrie Honeypot

Required system dependencies were installed:

```
sudo apt install python3-pip python3-dev libssl-dev libffi-dev build-essential
```

The Cowrie repository was cloned from the official GitHub source:

```
git clone https://github.com/cowrie/cowrie.git
```

4.3 Environment and Cowrie Setup

A Python virtual environment was created and activated:

```
python3 -m venv TESTenv
source TESTenv/bin/activate
```

The Python package manager (pip) was updated and the required dependencies were installed:

```
pip3 install --upgrade pip
```

```
pip3 install -r requirements.txt
```

After installation, the Ubuntu VM was switched back to the Host-Only Adapter to maintain the isolated environment.

The Cowrie configuration file was initialised: `cp etc/cowrie.cfg.dist etc/cowrie.cfg`

4.4 Cowrie Configuration

The default listening port was changed to allow connections from the Kali VM:

- Default: listen_endpoints = tcp:6415:interface=127.0.0.1
- Updated: listen_endpoints = tcp:2222:interface=0.0.0.0

4.5 Cowrie Execution and Monitoring

Cowrie was started using: `cowrie start`

Real-time log monitoring was performed using: `tail -f var/log/cowrie/cowrie.log`

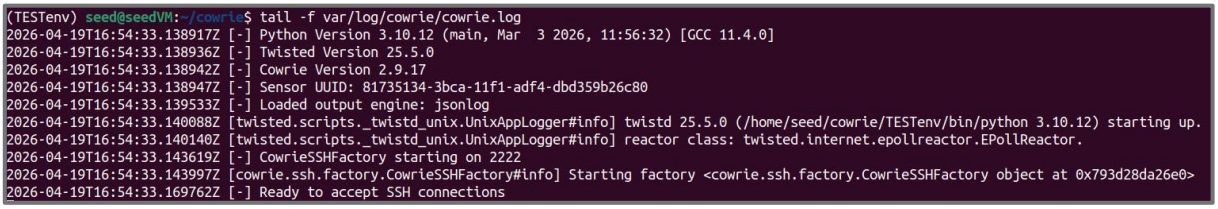

Figure 1 – Cowrie Execution

4.6 Connectivity Verification

Note: IP addresses and port numbers have been masked in this document. The values shown are placeholders representing the actual testing environment used.

Network configuration:

- Ubuntu IP: 192.xxx.xx.xxx
- Kali IP: 192.xxx.xx.xxx

The SSH connection was tested from Kali using: `ssh root@192.xxx.xx.xxx -p xxxx`

Connection attempts were then verified through the Cowrie logs.

5. Limitations & Risks

- Misconfigured VM networking could expose the honeypot outside the environment.
- Lack of monitoring beyond logs might make it difficult to catch issues during long-running tests.
- Simple setup that will not work on advanced attackers, reducing realism and accuracy of the results.

6. Testing & Findings

6.1 Automated Password Guessing Using Hydra

A password-guessing attack was simulated using a small password list:

- 123
- password
- admin
- root
- test

The attack was executed using Hydra: `hydra -l root -P passwords.txt ssh://192.xxx.xx.xxx -s xxxx`

Findings

```
New connection: 192.xxx.xx.xxx:32850
New connection: 192.xxx.xx.xxx:32864
New connection: 192.xxx.xx.xxx:32866
New connection: 192.xxx.xx.xxx:32888
New connection: 192.xxx.xx.xxx:32896
login attempt [b'root'/b'password'] succeeded
login attempt [b'root'/b'admin'] succeeded
login attempt [b'root'/b'test'] succeeded
login attempt [b'root'/b'123'] succeeded
login attempt [b'root'/b'root'] failed
```

- Multiple rapid connection attempts were observed from the attacker IP (192.xxx.xx.xxx), indicating automated brute-force activity.
- Various password combinations were tested against the root account.
- Several login attempts were successfully accepted by the honeypot due to the use of weak credentials.

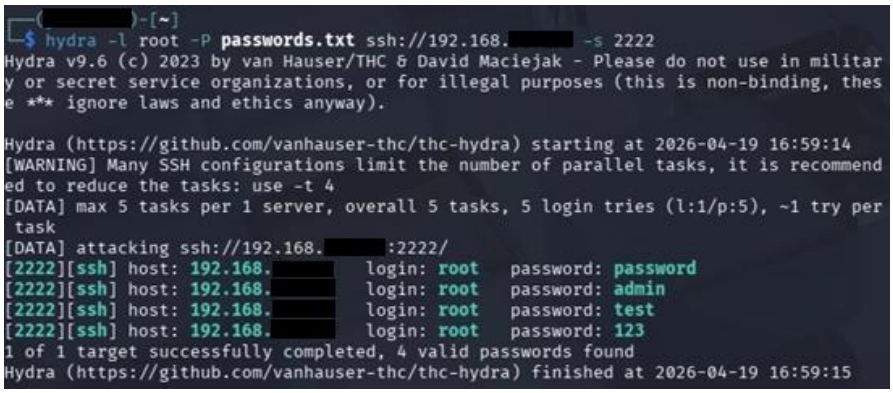

Figure 2 – Attacker: password brute-force

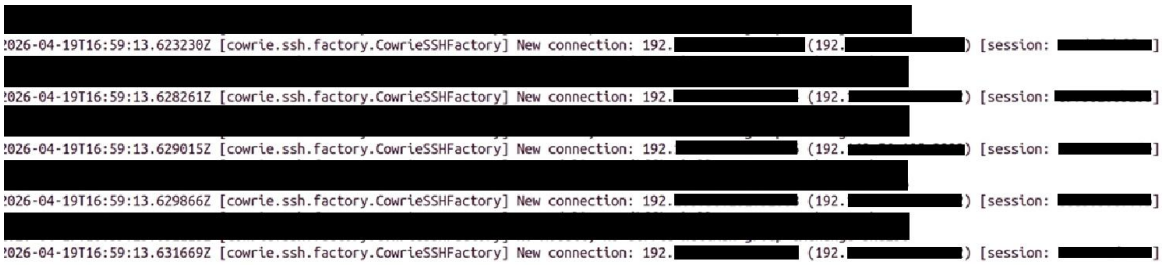

Figure 3 – Logs: connections

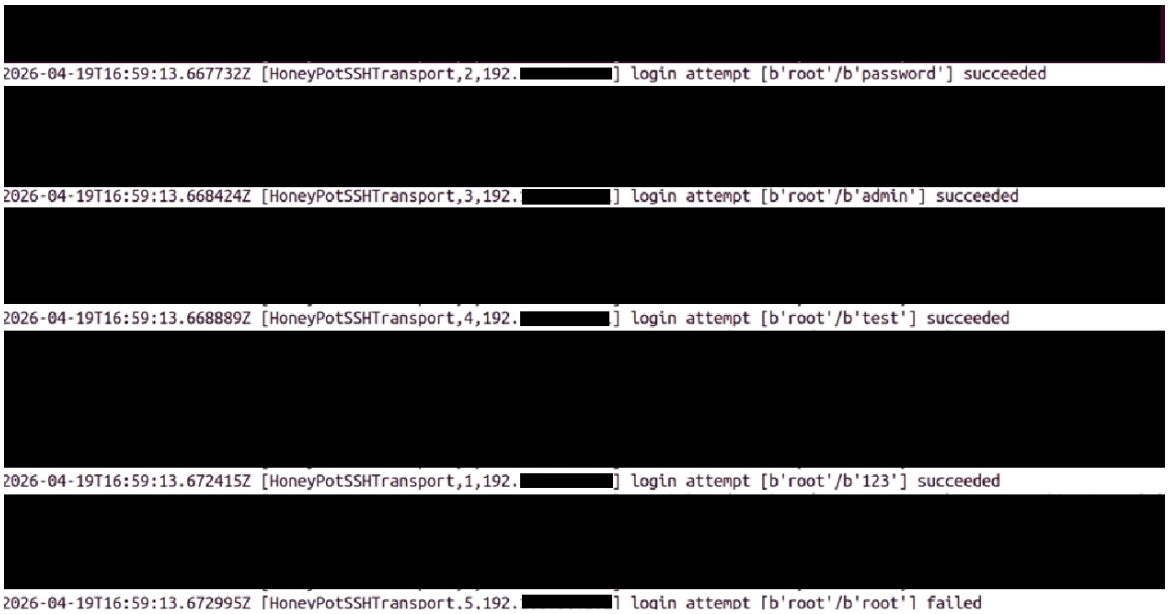

Figure 4 – Logs: login attempts

6.2 Manual Common-Credential Testing

Manual login attempts were performed using commonly used usernames:

- root: `ssh root@xxx.xxx.xx.xxx -p xxxx`
- admin: `ssh admin@xxx.xxx.xx.xxx -p xxxx`
- user: `ssh user@xxx.xxx.xx.xxx -p xxxx`

Each account was tested with the following passwords:

Username	Password Tested
root	root, test, password
admin	admin, 123, password
user	user, 123, password

Findings

```
New connection: 192.xxx.xx.xxx:xxxxx
[session: xx7aec83a5ae]

login attempt [b'root'/b'root'] failed
login attempt [b'root'/b'test'] succeeded

New connection: 192.xxx.xx.xxx:xxxxx
[session: c8dee3d11472]

login attempt [b'admin'/b'admin'] failed
login attempt [b'admin'/b'123'] failed
login attempt [b'admin'/b'password'] failed

New connection: 192.xxx.xx.xxx:xxxxx
[session: 22328958558e]

login attempt [b'user'/b'user'] failed
login attempt [b'user'/b'123'] failed
login attempt [b'user'/b'password'] failed
```

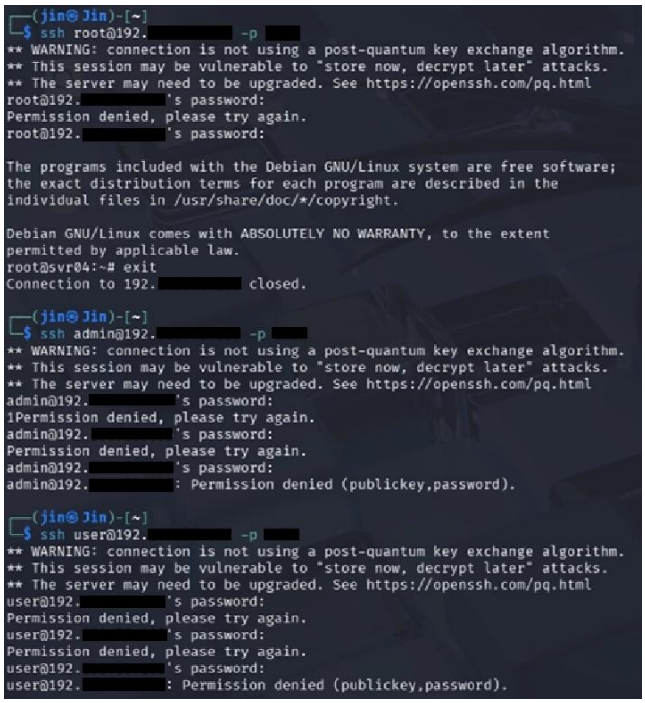

Figure 5 – Attacker: login attempts

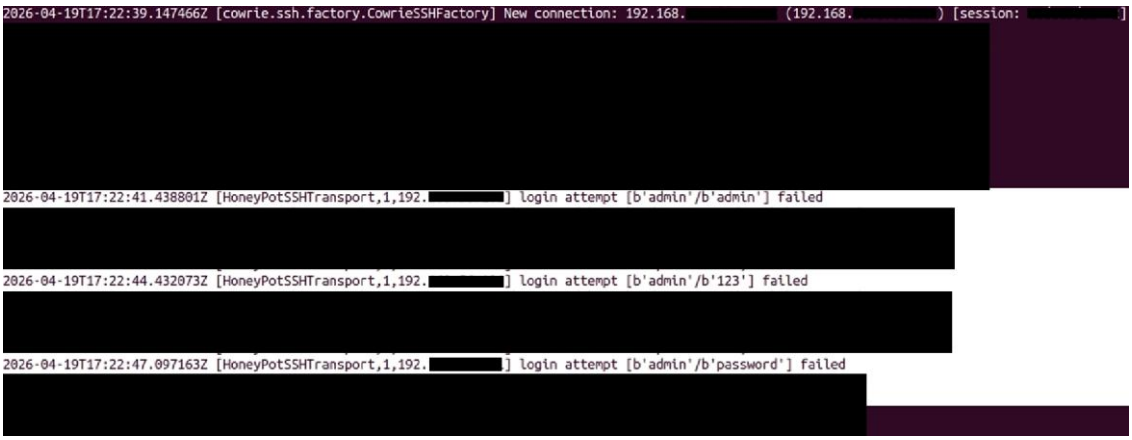

Figure 6, 7 & 8 – Logs: login attempts by attacker

6.3 Successful Login and Command Execution

A successful login was performed using weak credentials:

```
ssh root@192.xxx.xx.xxx -p xxxx
Password: test
```

After gaining access, the following commands were executed:

Command	Observation
`uname -a`	Retrieves basic system information.
`whoami`	Identifies the current user and privilege level.
`ls`	Lists files in the current directory.
`cat /etc/passwd`	Displays user account information.

This test demonstrates attacker behaviour after gaining access to a system.

Findings

```
New connection: 192.xxx.xx.xxx:xxxxx
[session: 0bc72c71e1cb]

2026-04-19T17:38:28.860277Z
[HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: uname -a

2026-04-19T17:38:31.849205Z
[HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: whoami

2026-04-19T17:38:33.590691Z
[HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: ls

2026-04-19T17:38:37.810867Z
[HoneyPotSSHTransport,0,192.xxx.xx.xxx] CMD: cat /etc/passwd
```

A successful connection was established by the attacker IP (192.xxx.xx.xxx). The commands demonstrate typical attacker reconnaissance behaviour.

Figure 9 – Attacker: command execution

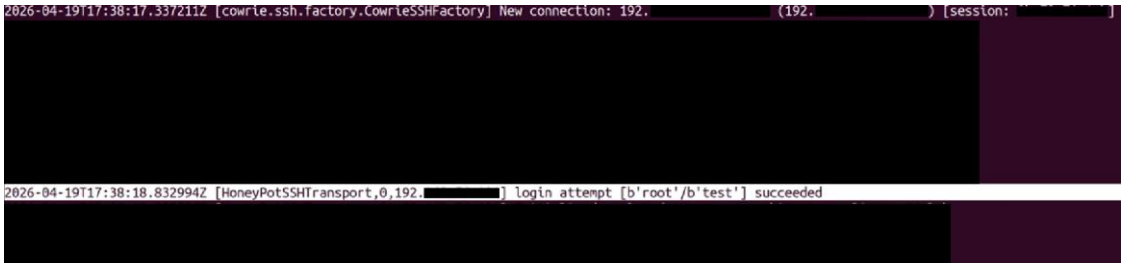

Figure 10 – Logs: login successful

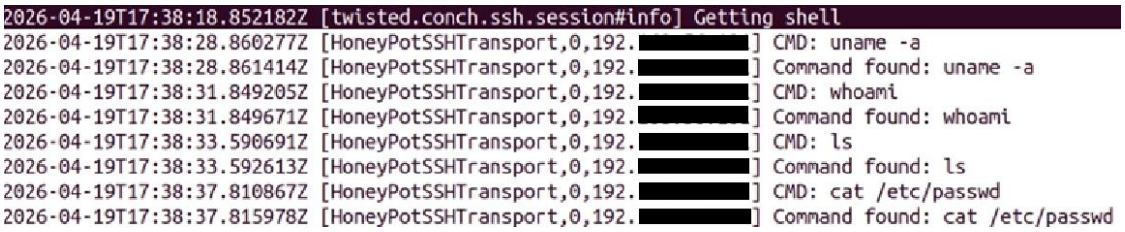

Figure 11 – Logs: command execution

6.4 Test Summary

Test	Result	Main Findings
Automated Password Guessing Using Hydra	Passed	Rapid automated login attempts were captured by Cowrie.
Manual Common-Credential Testing	Passed	`root/test` was accepted while the tested `admin` and `user` credentials failed.
Successful Login and Command Execution	Passed	Cowrie captured post-login commands and attacker interaction.

7. Challenges

Challenge	Resolution
Initial lack of planning and understanding resulted in constant switching between NAT and Host-Only Adapter modes to install required dependencies.	Improved understanding of when each network configuration was required during setup.
Shared clipboard functionality between the host and Ubuntu VM did not initially work, affecting workflow efficiency.	Resolved by installing VirtualBox Guest Additions and enabling bidirectional clipboard support.
Dependency installation issues were encountered due to environment inconsistencies and package version conflicts.	Resolved by using a clean Python virtual environment and updating pip.
Difficulty setting up the Cowrie directory and configuration, including the missing `cowrie.cfg` file.	Resolved by copying the default configuration template.

8. Improvements

- Improve initial planning to reduce the need for switching between different network configurations, such as NAT and Host-Only modes.
- Implement automated log analysis, such as real-time log visualisation, to filter relevant information and identify patterns in attack behaviour for more efficient analysis.
- Improve the honeypot to include additional services such as HTTP or FTP to capture a wider range of attack behaviours.
- Configure a custom user credentials database to simulate more realistic login scenarios and improve the accuracy of captured interactions.